# MongoDB CRUD Operations using Command Prompt

## 1. Database &collection Creation

```
Microsoft Windows [Version 10.0.17763.1]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\ELCOT>mongosh
Current Mongosh Log ID: 69f8dfa529374e79e4abc113
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:          7.0.32
Using Mongosh:          2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-05-03T23:40:46.164+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------

test> show dbs
admin      40.00 KiB
config     84.00 KiB
libraryDB  72.00 KiB
local      72.00 KiB
school     40.00 KiB
studentDB  120.00 KiB
test>  use studentDB
switched to db studentDB
studentDB> db.createCollection("students")
{ ok: 1 }
```

# 2. Insert operations

```
studentDB> db.students.insertOne({name:"Arun",age:21,course:"MERN Stack",status:"ongoing"})
{
  acknowledged: true,
  insertedId: ObjectId('69f8e05a29374e79e4abc114')
}
studentDB> db.students.insertMany([{name:"Kavi",age:22,course:"MERN Stack", status:"ongoing"},{name:"Ravi",age:23,course:"Java",status:"completed"},{name:"Meena",age:20
,course:"MERN Stack",status:"ongoing"}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f8e18829374e79e4abc115'),
    '1': ObjectId('69f8e18829374e79e4abc116'),
    '2': ObjectId('69f8e18829374e79e4abc117')
  }
}
studentDB> db.students.find()
[
  {
    _id: ObjectId('69f7a585d582aa9cccabc114'),
    name: 'Abishek',
    age: 21,
    course: 'Javascript',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f7a8997d68e5ed9dabc115'),
    name: 'Nanthitha',
    age: 20,
    course: 'MERN STACK',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f7a8997d68e5ed9dabc116'),
    name: 'Dharshini',
    age: 20,
    course: 'Full Stack Developer',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f7a8997d68e5ed9dabc117'),
    name: 'abi',
    age: 22,
    course: 'HTML',
    status: 'finished'
```

# 3. Read operations

```
studentDB> db.students.find()
[
  {
    _id: ObjectId('69f7a585d582aa9cccabc114'),
    name: 'Abishek',
    age: 21,
    course: 'Javascript',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f7a8997d68e5ed9dabc115'),
    name: 'Nanthitha',
    age: 20,
    course: 'MERN STACK',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f7a8997d68e5ed9dabc116'),
    name: 'Dharshini',
    age: 20,
    course: 'Full Stack Developer',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f7a8997d68e5ed9dabc117'),
    name: 'abi',
    age: 22,
    course: 'HTML',
    status: 'finished'
  },
  {
    _id: ObjectId('69f8cf4bfc382469e9abc114'),
    name: 'Arun',
    age: 21,
    course: 'Java',
    status: 'completed'
  },
  {
    _id: ObjectId('69f8d1c0296b5e1117abc114'),
    name: 'Arun',
    age: 21,
    course: 'Java',
    status: 'ongoing'
  },
```

# 4. Update Operations

```
studentDB> db.students.find({course:"MERN Stack"})
[
  {
    _id: ObjectId('69f8d299296b5e1117abc115'),
    name: 'nanthitha',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f8e05a29374e79e4abc114'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f8e18829374e79e4abc115'),
    name: 'Kavi',
    age: 22,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f8e18829374e79e4abc117'),
    name: 'Meena',
    age: 20,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
studentDB> db.students.updateOne({namr:"Arun"},{$set:{status:"completed"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
studentDB> db.students.updateMany({course:"MERN Stack"},{$set:{status:"active"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 4,
```

# 5. Delete Operations

```
studentDB> db.students.updateOne({namr:"Arun"},{$set:{status:"completed"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
studentDB> db.students.updateMany({course:"MERN Stack"},{$set:{status:"active"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 4,
  modifiedCount: 4,
  upsertedCount: 0
}
studentDB> db.students.deleteOne({name:"Ravi"})
{ acknowledged: true, deletedCount: 1 }
studentDB> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 11 }
```

6. Query Operations:

```
studentDB> db.students.find({age:{$gt:21}})

studentDB> db.students.find({age:{$lt:22}})

studentDB> db.students.find({course:{$in:["MERN Stack","Java"]}})

studentDB> db.students.find({$and:[{course:"MERN Stack"},{status:"ongoing"}]})

studentDB> db.students.find({$or:[{age:20},{course:"Java"}]})

studentDB> db.student.find({status:{$exists:True}})
ReferenceError: True is not defined
studentDB> db.student.find({status:{$exists:true}})
```

# 7. Use Case: Library System

```
studentDB> use libraryDB
switched to db libraryDB
libraryDB> db.createCollection("books")
{ ok: 1 }
libraryDB> db.books.insertMany([{name:"MongoDB Basics",author:"john",available:true,price:500},{name:"Node.js Guide",author:"Alex",available:false,price:700},{name:"Rea
ct Handbook",author:"Sara",available:true, price:600}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f8e66029374e79e4abc118'),
    '1': ObjectId('69f8e66029374e79e4abc119'),
    '2': ObjectId('69f8e66029374e79e4abc11a')
  }
}
libraryDB> db.books.find({available:true})
[
  {
    _id: ObjectId('69f7ad8a7d68e5ed9dabc119'),
    title: 'Node.js Guide',
    author: 'Alex',
    available: true,
    price: 100
  },
  {
    _id: ObjectId('69f8e66029374e79e4abc118'),
    name: 'MongoDB Basics',
    author: 'john',
    available: true,
    price: 500
  },
  {
    _id: ObjectId('69f8e66029374e79e4abc11a'),
    name: 'React Handbook',
    author: 'Sara',
    available: true,
    price: 600
  }
]
libraryDB> db.books.updateOne({name:"Node.js Guide"},{$set:{available:true}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1
```

```
libraryDB> db.books.updateOne({name:"Node.js Guide"},{$set:{available:true}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
libraryDB> db.books.deleteOne({name:"MongoDB Basics"})
{ acknowledged: true, deletedCount: 1 }
libraryDB> db.books.find({price:{$gt:500}})
[
  {
    _id: ObjectId('69f7ad8a7d68e5ed9dabc11a'),
    title: 'React Handbook',
    author: 'Sara',
    available: false,
    price: 800
  },
  {
    _id: ObjectId('69f8e66029374e79e4abc119'),
    name: 'Node.js Guide',
    author: 'Alex',
    available: true,
    price: 700
  },
  {
    _id: ObjectId('69f8e66029374e79e4abc11a'),
    name: 'React Handbook',
    author: 'Sara',
    available: true,
    price: 600
  }
]
libraryDB>
```

Conclusion:

Mongodb CRUD Operations were successfully performed using Mongosh